

SAS Progress Console

Projet final de synthèse : réunir les notions des sept journées dans une application JavaScript exécutée dans la console.

CADRE	ATTENDU
Environnement	JavaScript avec Node.js, sans framework ni dépendance externe.
Données	Des apprenants fictifs et leurs résultats d'exercices.
Preuves à remettre	Un programme exécutable, un README, des tests et un historique Git.

La mission

Ta mission finale consiste à réunir les notions des sept journées du SAS dans une seule solution cohérente. Le tableau ci-dessous indique les preuves attendues pour chaque notion.

L'application doit gérer des apprenants fictifs et leurs résultats journaliers, calculer leur progression et afficher un **tableau de bord pédagogique** permettant à un formateur de repérer ce qui est acquis et ce qui reste à travailler.

Ce que tu dois démontrer

- Analyser un besoin et découper un problème en tâches simples.
- Construire des fonctions claires et des données bien organisées.
- Tester le fonctionnement, corriger les erreurs et expliquer tes choix.

L'objectif n'est pas d'écrire beaucoup de code, mais de produire une solution compréhensible, correcte et maîtrisée.

Une aide pédagogique, jamais une décision automatique

Le programme ne doit **jamais décider si une personne est admise ou non**. Les niveaux affichés décrivent uniquement les données du projet.

Lien avec les sept journées du SAS

NOTION	PREUVE ATTENDUE DANS LE PROJET
Variables, types, opérateurs et conditions	Validation des entrées et calcul des indicateurs.
Boucles	Parcours des apprenants et de leurs résultats.
Fonctions	Découpage en responsabilités simples.
Chaînes de caractères	Nettoyage et recherche des noms.
Tableaux	Stockage, filtre et transformation des listes.
Objets	Modèle d'un apprenant et d'un résultat journalier.
Recherche et tri	Recherche par nom, classement par progression et tri alphabétique.

YOUCODE / SAS JAVASCRIPT / PARTIE 02

Les données de départ

Utilise uniquement des personnes fictives. Tu peux reprendre cette structure et l'enrichir selon les besoins de ta solution.

```
const apprenants = [
  {
    id: 1,
    nomComplet: "Sara Dev",
    ville: "Nador",
    resultats: [
      { jour: 1, exercicesTerminees: 18,
        totalExercices: 20, challengeTermine: true },
      { jour: 2, exercicesTerminees: 14,
        totalExercices: 20, challengeTermine: false }
    ]
  },
  {
    id: 2,
    nomComplet: "Yassine Code",
    ville: "Oujda",
    resultats: [
      { jour: 1, exercicesTerminees: 12,
        totalExercices: 20, challengeTermine: false }
    ]
  }
];
```

Comprendre la structure

CHAMP	RÔLE
id	Identifiant de l'apprenant. Deux apprenants ne peuvent pas partager le même identifiant.
nomComplet / ville	Informations fictives de présentation. Le nom est nettoyé et normalisé pour la recherche.
resultats	Tableau contenant les résultats des journées renseignées.
jour	Numéro de la journée, compris entre 1 et 7.
exercicesTerminees / totalExercices	Nombre d'exercices terminés et nombre d'exercices proposés pour cette journée.
challengeTermine	Booléen indiquant si le challenge de la journée a été terminé.

Un résultat absent correspond à une journée non renseignée. Un challenge marqué `false` correspond à un challenge non terminé pour une journée déjà renseignée.

Les fonctionnalités essentielles

Les entrées doivent être vérifiées avant d'être enregistrées. Les indicateurs doivent ensuite être calculés directement à partir des données.

1. Nettoyer et valider

- Supprimer les espaces inutiles dans les noms et uniformiser leur écriture pour faciliter la recherche.
- Vérifier que le numéro du jour est compris entre **1 et 7**.
- Vérifier que les nombres d'exercices sont valides et cohérents.
- Refuser toute opération incorrecte avec un message compréhensible.

Exemple de cohérence : le nombre d'exercices terminés ne peut pas dépasser le nombre d'exercices proposés. Un message d'erreur doit expliquer ce qui pose problème.

2. Gérer les apprenants et leurs résultats

- Ajouter un apprenant sans accepter deux fois le même identifiant.
- Ajouter le résultat d'une journée ou mettre à jour celui qui existe déjà.
- Retrouver un apprenant à partir de son identifiant.
- Rechercher un apprenant avec tout ou partie de son nom, sans tenir compte des majuscules.

3. Calculer la progression individuelle

INDICATEUR	CALCUL / INTERPRÉTATION
Exercices terminés	Somme des exercices terminés sur les journées renseignées.
Exercices proposés	Somme des exercices proposés sur les journées renseignées.
Progression	$(\text{Total terminé} / \text{total proposé}) \times 100$.
Challenges terminés	Nombre de résultats dont le challenge est terminé.
Journées renseignées	Nombre de journées pour lesquelles un résultat est enregistré.

Exemple avec Sara Dev : $18 + 14 = \mathbf{32 \text{ exercices terminés}}$ sur 40 proposés, soit **80 %**, avec 1 challenge terminé et 2 journées renseignées. Pense à traiter explicitement un total proposé nul pour éviter une division par zéro.

Afficher un repère pédagogique

NIVEAU	SEUIL DU DOCUMENT SOURCE
Solide	À partir de 80 %.
En progression	Entre 50 % et 79 %.
À renforcer	En dessous de 50 %.

Si tu utilises des pourcentages décimaux, documente et teste ta convention d'arrondi ou de classement aux frontières des seuils.

À retenir

Ces repères décrivent les résultats enregistrés. **Ils ne constituent pas une décision d'admission.**

Explorer et restituer les résultats

La recherche, le filtrage et le tri rendent les données utiles. Le tableau de bord rassemble les informations nécessaires à la lecture de la progression.

4. Rechercher, filtrer et trier

- Rechercher un apprenant par tout ou partie de son nom.
- Filtrer les apprenants selon leur niveau de progression.
- Trier les apprenants par pourcentage, **du plus élevé au plus faible**.
- Permettre également un tri alphabétique.

5. Afficher le tableau de bord console

Le tableau de bord doit afficher au minimum :

- Le nombre total d'apprenants et la progression moyenne du groupe.
- Le nombre de profils **Solide**, **En progression** et **À renforcer**.
- La liste triée des apprenants avec leur pourcentage.
- Les journées et les challenges manquants pour chaque apprenant.

Pour une lecture claire, distingue les journées non renseignées des challenges non terminés. La méthode de calcul de la moyenne du groupe et le traitement des données absentes doivent être expliqués dans le README.

Fonctions minimales attendues

Les noms peuvent être adaptés. Chaque fonction doit avoir une responsabilité claire.

FONCTION	RESPONSABILITÉ
normaliserNom	Nettoyer et uniformiser un nom.
validerResultat	Vérifier les valeurs d'un résultat journalier.
ajouterApprenant	Ajouter un apprenant en contrôlant les doublons d'identifiant.
enregistrerResultat	Ajouter ou mettre à jour une journée.
rechercherApprenant	Retrouver un profil par identifiant ou par nom.
calculerProgression	Produire les indicateurs individuels.
filtrerParNiveau	Sélectionner les profils d'un niveau donné.
trierParProgression	Classer les profils par progression décroissante.
afficherTableauDeBord	Présenter les indicateurs du groupe et les listes.

Le menu de l'application

Un exemple d'interface console pour visualiser les actions disponibles et comprendre le parcours de l'utilisateur.

Menu principal proposé

```
SAS PROGRESS CONSOLE
1. Afficher le tableau de bord
2. Afficher la liste des apprenants
3. Ajouter un apprenant
4. Consulter un apprenant par identifiant
5. Ajouter ou modifier le résultat d'une journée
6. Rechercher un apprenant par nom
7. Filtrer les apprenants par niveau
8. Trier les apprenants par progression décroissante
9. Trier les apprenants par ordre alphabétique
0. Quitter
```

Votre choix :

Les numéros et les libellés peuvent être adaptés. Ce menu illustre les fonctionnalités du projet ; il ne fournit pas leur code de réalisation.

L'ajout demande un **identifiant, un nom complet et une ville**. La consultation utilise l'identifiant ; la recherche accepte un nom partiel. Le filtre propose les trois niveaux existants. L'enregistrement d'une journée suit les questions de l'exemple ci-dessous.

Exemple d'utilisation : enregistrer une journée

Cet exemple reprend Sara Dev et les données de départ. Les valeurs tapées par l'utilisateur sont affichées après chaque question.

```
Votre choix : 5
Identifiant de l'apprenant : 1
Apprenant trouvé : Sara Dev
Jour (1 à 7) : 3
Exercices terminés : 16
Total d'exercices proposés : 20
Challenge terminé (oui/non) : oui
Résultat du jour 3 enregistré.
Sara Dev : 48 / 60 exercices, progression 80 %.
3 journées renseignées, 2 challenges terminés.
```

Comportement attendu du menu

- Lire le choix de l'utilisateur, lancer l'action correspondante, puis afficher un résultat ou un message clair.
- Après une action, revenir au menu pour permettre une nouvelle opération.
- Si le choix, l'identifiant ou les données sont invalides, expliquer l'erreur sans enregistrer de résultat incorrect.
- Le choix **0** termine le programme avec un message de fermeture.

YOUCODE / SAS JAVASCRIPT / PARTIE 06

Organiser, exécuter et tester

Une structure simple et des tests explicites permettent de vérifier la solution et de la présenter sereinement.

Contraintes techniques et de travail

- Utiliser **JavaScript avec Node.js**, sans framework et sans dépendance externe.
- Ne jamais utiliser de vraies données personnelles.
- Calculer les résultats affichés depuis les données ; ne pas les modifier manuellement.
- Prévoir **au moins cinq scénarios de test, dont deux cas invalides ou limites**.
- Pouvoir expliquer et modifier tout code utilisé.
- Faire des commits réguliers avec des messages compréhensibles.

Organisation conseillée

```
projet-final/  
|-- README.md  
|-- src/  
|   |-- data.js  
|   |-- progression.js  
|   `-- index.js  
`-- tests/  
    |-- scenarios.js
```

Une solution dans un seul fichier reste acceptable si elle est claire et bien découpée en fonctions.

Commandes proposées

```
node projet-final/src/index.js  
node projet-final/tests/scenarios.js  
git add projet-final  
git commit -m "projet final : construire le tableau de bord de progression"  
git push
```

Ces commandes reprennent l'organisation proposée. Exécute-les depuis le dossier qui contient projet-final et adapte les chemins si ta structure est différente.

Exemples de scénarios de test

Suggestions à adapter : elles illustrent les cinq scénarios demandés et ne constituent pas des fonctionnalités supplémentaires.

SCÉNARIO	RÉSULTAT À VÉRIFIER
1. Ajouter un profil valide	Le profil est ajouté et peut être retrouvé par identifiant.
2. Mettre à jour une journée	Le résultat existant est remplacé, sans dupliquer la journée.
3. Calculer et rechercher	Sara affiche 80 % ; une recherche partielle avec une casse différente la retrouve.
4. Identifiant déjà utilisé Cas invalide	L'ajout est refusé avec un message compréhensible.
5. Résultat incohérent Cas invalide	Un jour hors de 1 à 7, ou trop d'exercices terminés, est refusé.

Livrables et remise

La remise doit permettre au formateur de lancer le programme, de vérifier tes tests et de comprendre les étapes de construction.

Les livrables attendus

- Le dossier projet-final dans ton dépôt GitHub personnel.
- Un README.md expliquant le besoin, les fonctionnalités, le lancement et les scénarios testés.
- Un programme qui s'exécute avec Node.js sans erreur.
- Un historique Git montrant les étapes de construction du projet.

Contenu conseillé pour le README

Présente le besoin et la structure des données, liste les fonctionnalités réalisées, indique les commandes de lancement et décris les tests. Explique les conventions retenues pour les calculs, les données absentes et les cas limites.

Ce qui sera observé

- La logique et le fonctionnement réel de la solution.
- La maîtrise des notions travaillées pendant les sept journées.
- La capacité à tester, corriger et expliquer le code.
- La clarté du code et des commits.
- L'autonomie et la réaction face à une modification imprévue.

La décision pédagogique reste globale

Le projet final est une preuve importante. La décision pédagogique finale tient aussi compte du **travail quotidien** et de l'**historique Git**.

Vérification avant la remise

- Le lien du dépôt GitHub personnel est prêt à être transmis.
- Le dernier push est effectué avant l'heure limite indiquée dans Classroom.
- Le projet est exécutable et la documentation est présente.

Document préparé à partir de « Google Classroom - Projet final de synthèse ». La date et l'heure de remise restent celles communiquées dans Classroom.